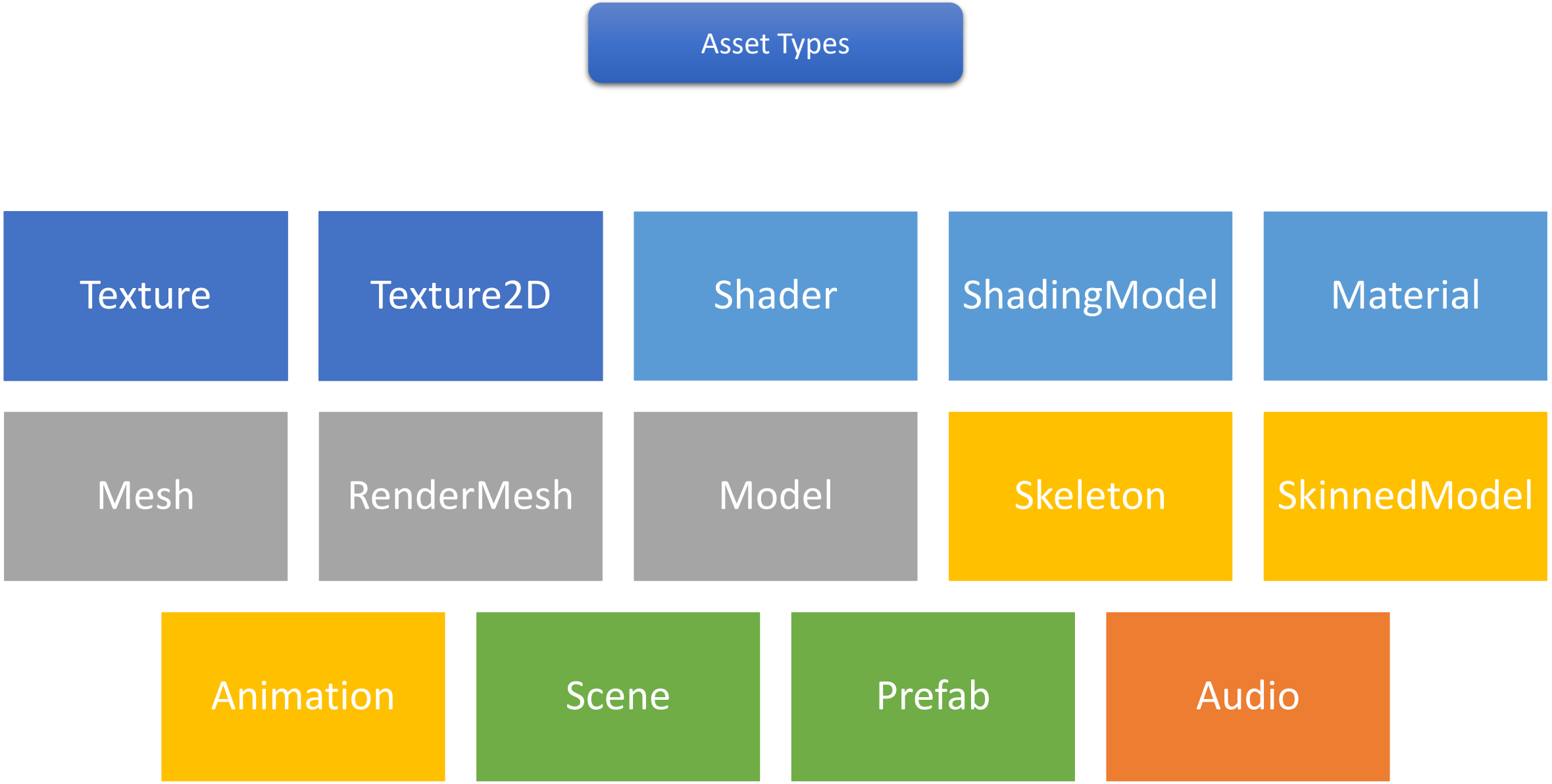
Asset Pipeline of Fools Engine

Work in progress


Asset Types
Texture
Texture2D
Shader
ShadingModel
Material
Mesh
RenderMesh
Model
Skeleton
SkinnedModel
Animation
Scene
Prefab
Audio

Import Model and On-Disk Structure

Asset's data is not saved to disk after importing. Loading and importing processes are unified in the editor. Import settings for a given asset are saved in an Asset file with metadata (eg. Texture resolution). Some assets have only Asset files and no Asset Source files (eg. Materials).

Setting import options is necessary only once (unless a change in the Asset Source File creates a need for adjustment).
Assets can be hot-reloaded at the editor's runtime or the runtime of a non-distribution game build without manual reimporting.
Asset data is stored on disk in one form without duplication.
Asset's metadata are always loaded and available.

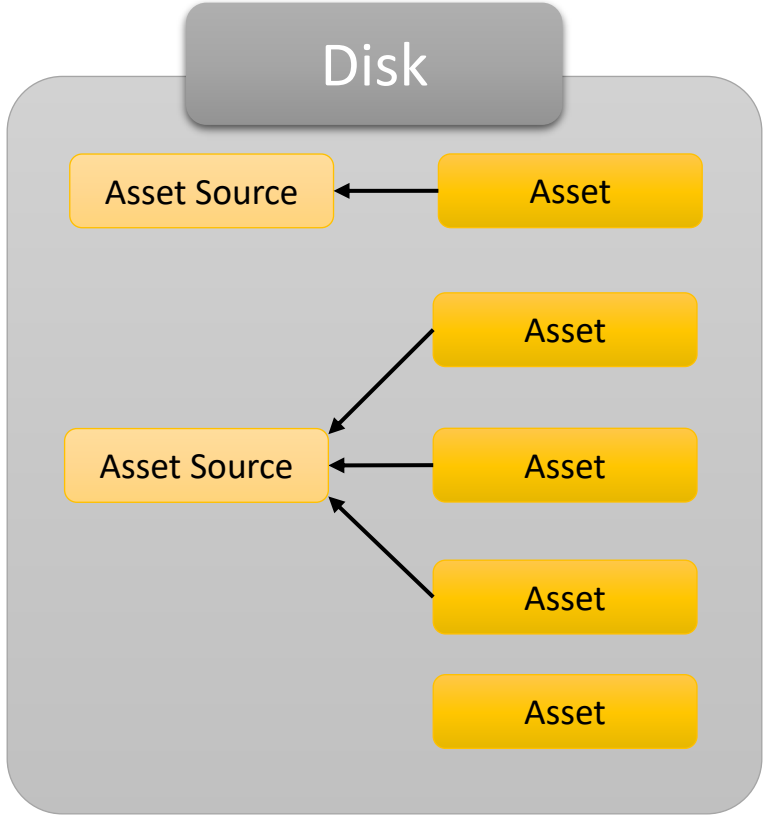
Examples

Texture

```
UUID: 7533288542743074609
Source Filepath: Survival_BackPack_2\1001_albedo.jpg
Usage: Map_BaseColor
Components: RGB
Format: RGB_8
Width: 4096
Height: 4096
```

Mesh

```
Source Filepath: Survival_BackPack_2\Survival_BackPack_2.fbx
Vertex Count: 47534
Index Count: 203724
```



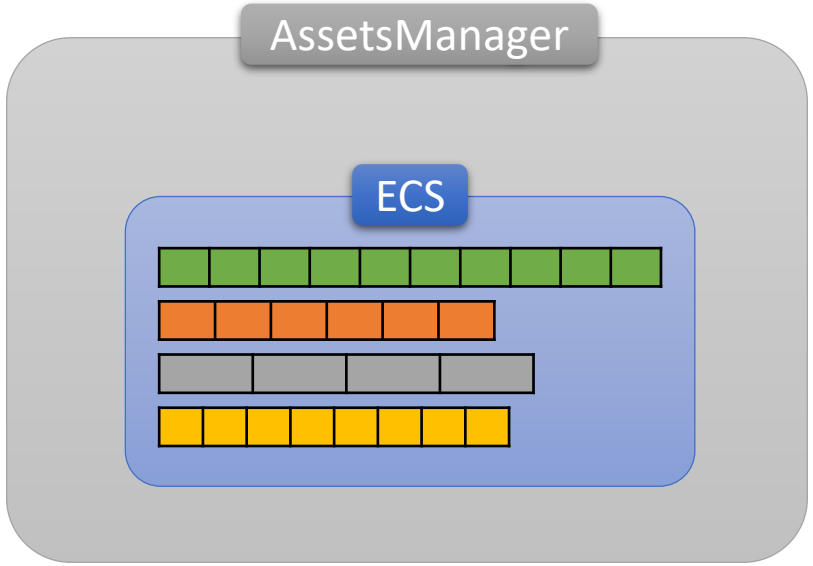
Runtime structure
Editor and Game

Assets are broken down into components and stored in ECS.
Components use AC prefix (from **A**sset **C**omponent).

Common Components:
- ACAssetType
- ACUUID
- ACFilepath
- ACSourceFilepath
- ACMasterAsset
- ACRefsCounters

Asset does not have to have all of those components (more on that later).

Every asset has a CoreComponent, that is different for every Asset type.
It contains metadata about the asset and a pointer to the data in CPU memory.
CPU side representation of GPU stored copy of the asset is also a component.



Asset Archetypes

Project Asset

- Assets that the game developer operates on.
- Components:
  - ACAssetType
  - ACUUID
  - ACFilepath
  - ACSourceFilepath (if needed)
  - ACRefsCounters
  - CoreComponent (asset's meta data and ptrs to data)
  - other, asset type specific components

Internal Asset

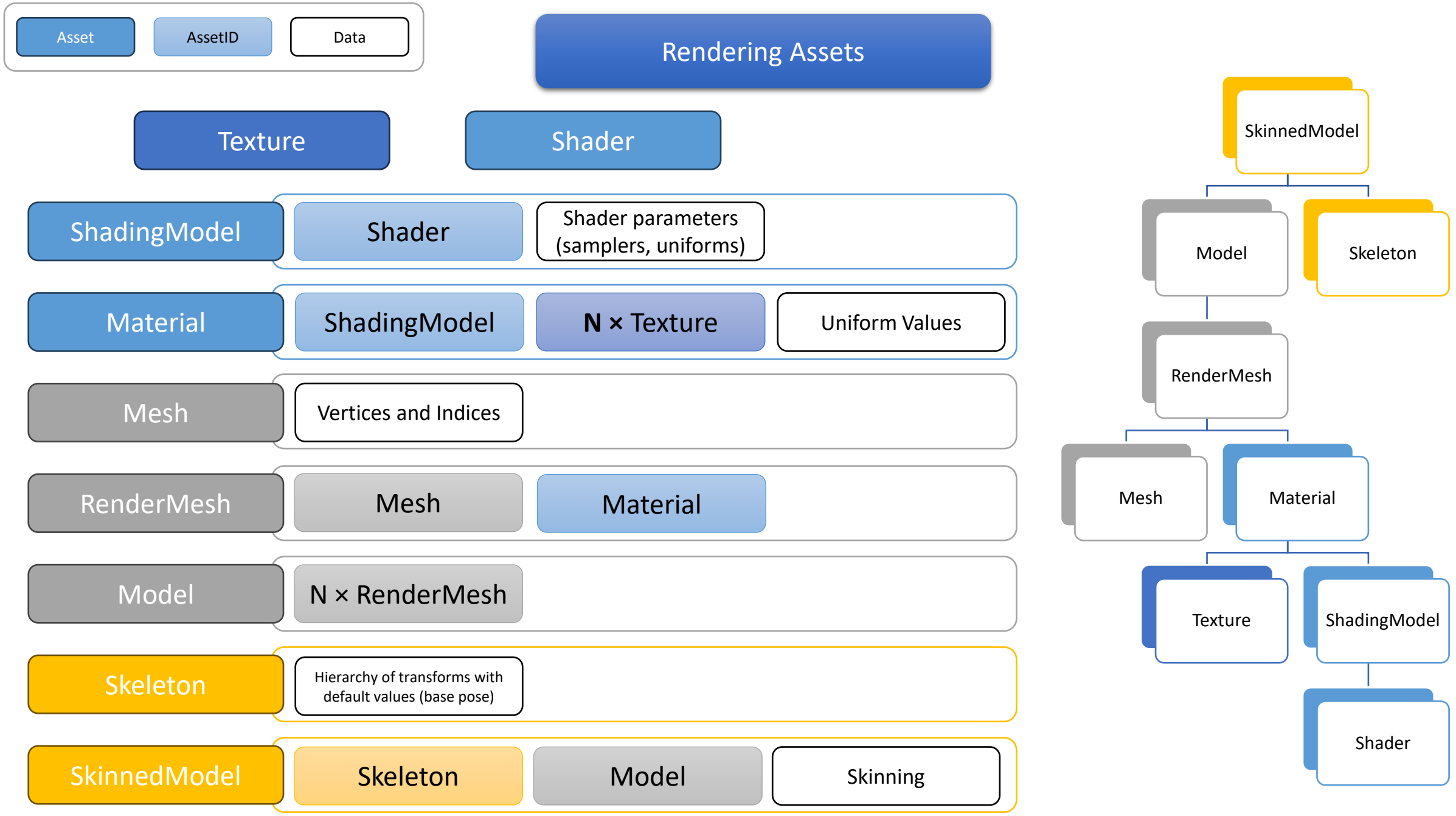
- Internal Assets are only ever referenced by other assets. They are conceptually parts of Project Assets and they get imported and loaded together. They are fully hidden from the user.
- Components:
  - ACAssetType
  - ACUUID
  - ACMasterAsset (ref back to Project Asset)
  - CoreComponent
  - other, asset type specific components

Editor Asset

- Assets used by only by editor (e.g. icons).
- Components
  - ACAssetType
  - CoreComponent
  - other, asset type specific components

Base Asset

- Assets build-in to the engine used by default.
- Components
  - ACAssetType
  - ACUUID
  - ACFilepath
  - CoreComponent
  - other, asset type specific components


Asset
AssetID
Data
Rendering Assets
Texture
Shader
ShadingModel
Shader
Shader parameters (samplers, uniforms)
Material
ShadingModel
N × Texture
Uniform Values
Mesh
Vertices and Indices
RenderMesh
Mesh
Material
Model
N × RenderMesh
Skeleton
Hierarchy of transforms with default values (base pose)
SkinnedModel
Skeleton
Model
Skinning
SkinnedModel
Model
Skeleton
RenderMesh
Mesh
Material
Texture
ShadingModel
Shader

Scene Components for Geometry Assets

Geometry assets (sufficient for rendering on their own) have dedicated scene components.

CRenderMesh	CModel	CSkinnedModel
• AssetHandle<RenderMesh>	• AssetHandle<Model>	• AssetHandle<SkinnedModel>

They also have scene components “simulating” them – asset views. This allows for runtime dynamic composition.

CRenderMeshView	CModelView	CSkinnedModel
• AssetHandle<Mesh> • AssetHandle<Material>	• Vector<AssetHandle<RenderMesh>>	• AssetHandle<Skeleton> • Vector<AssetHandle<RenderMesh>> • Skinning

Runtime Management
Editor and Game

You can never assume an asset is loaded and there is no explicit asset loading request possibility. Instead, a handle to the asset has a loading priority value - enum.

Asset handles of each loading priority are atomically counted in a component of that asset (ACRefsCounters).
List of assets to load for given loading priority is maintained continuously by adding/removing empty component corresponding to given loading priority when reference count changes from/to 0.

Asset loading/unloading runs in an asynchronous loop, but in a foreground (structured concurrency).

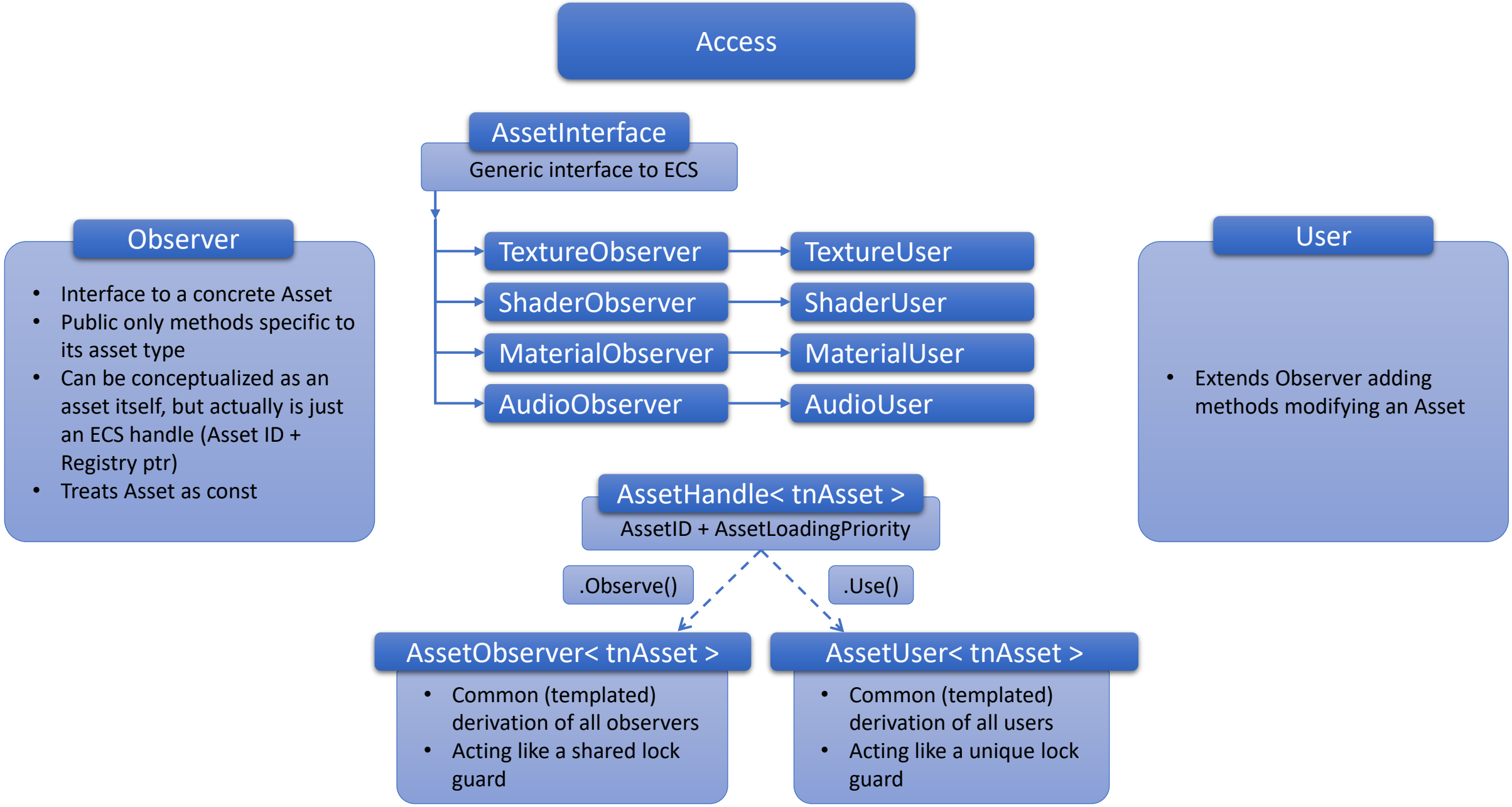
This makes dynamic adaptive asset loading management much easier and safer (e.g. preloading assets for next level simply by setting their handles from None to Low). There is no need for the user to write asynchronous multithreaded code and still get it's benefits.

Load / unload lists

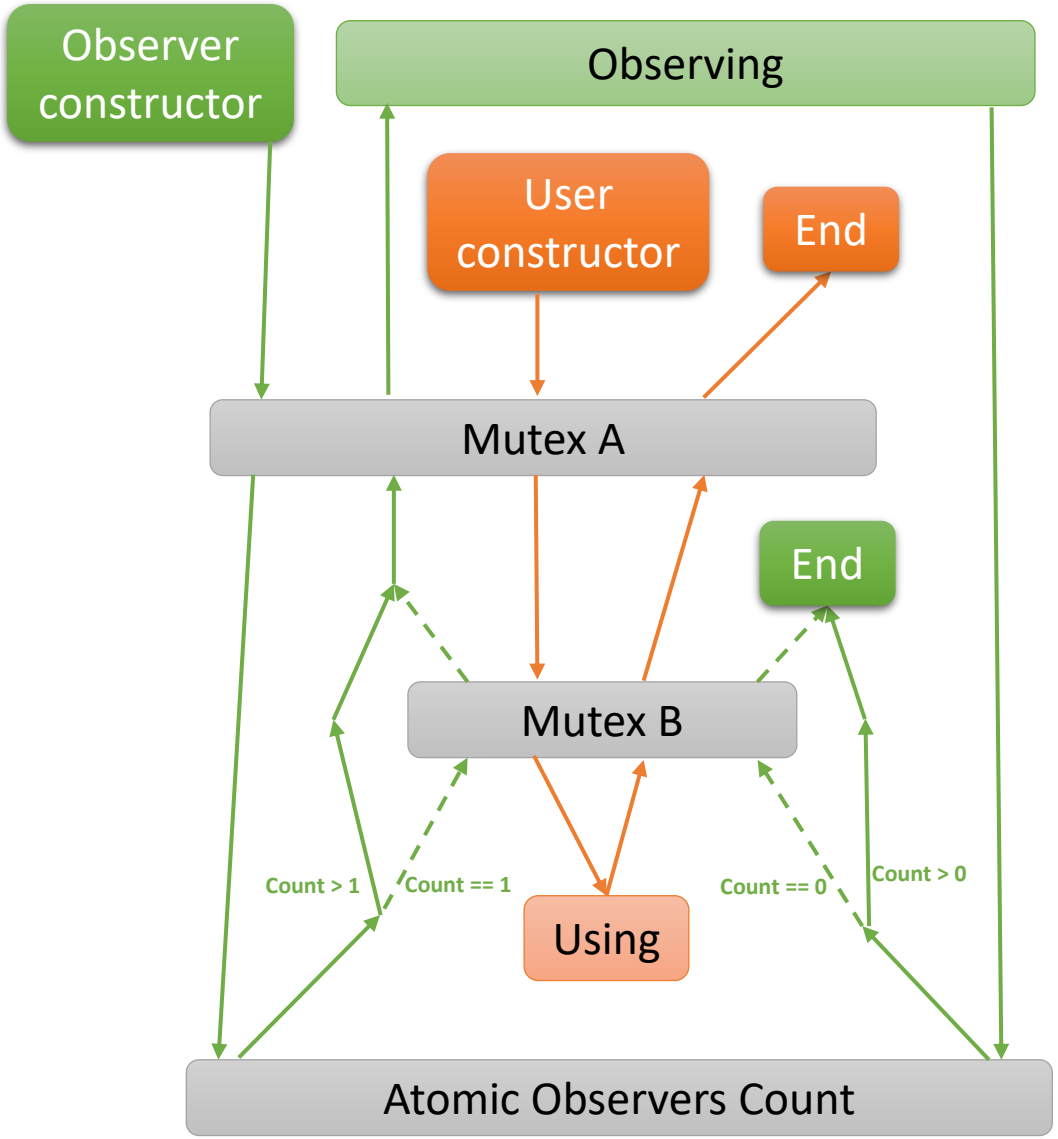
```
m_LoadingGroups.Unload = m_Registry.group<>(
    entt::get<ACLoadedFlag>,
    entt::exclude<
        ACLoadedAsDependence,
        ACLoadFlag<AssetLoadingPriority::Minimal>,
        ACLoadFlag<AssetLoadingPriority::VeryLow>,
        ACLoadFlag<AssetLoadingPriority::Low>,
        ACLoadFlag<AssetLoadingPriority::Standard>,
        ACLoadFlag<AssetLoadingPriority::High>,
        ACLoadFlag<AssetLoadingPriority::VeryHigh>,
        ACLoadFlag<AssetLoadingPriority::Critical>
    >
);

m_LoadingGroups.Minimal = m_Registry.group<>(
    entt::get<ACLoadFlag<AssetLoadingPriority::Minimal>>,
    entt::exclude<ACLoadedFlag>
);

m_LoadingGroups.VeryLow = m_Registry.group<>(
    entt::get<ACLoadFlag<AssetLoadingPriority::VeryLow>>,
    entt::exclude<ACLoadedFlag>
);

m_LoadingGroups.Low = m_Registry.group<>(
    entt::get<ACLoadFlag<AssetLoadingPriority::Low>>,
```

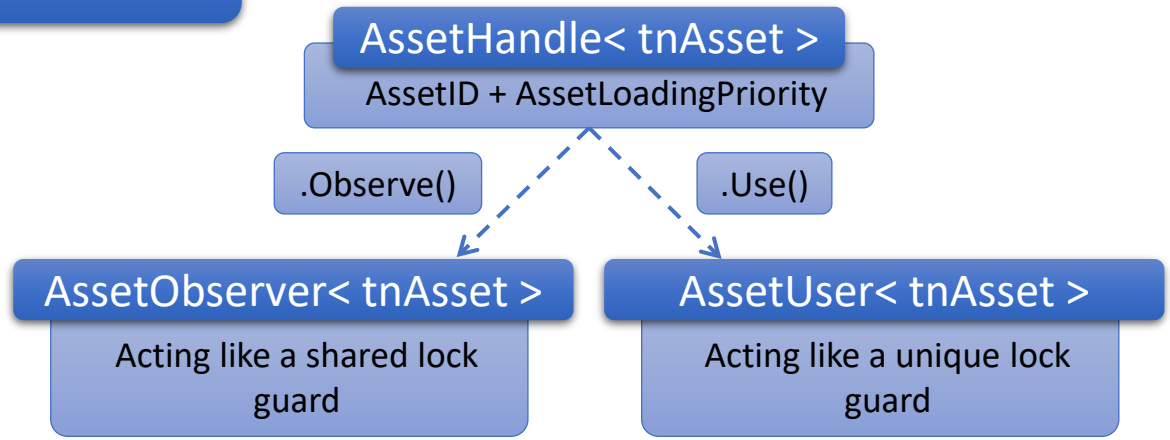
AssetHandle

```
void Init()
{
    if (!m_ID) return;
    if (m_LoadingPriority == AssetLoadingPriority::None) return;
    auto ECSHandle = GetECSHandle();
    auto refs = ECSHandle.try_get<ACRefsCounters>();
    if (!refs) return;
    if (refs->LiveHandles[m_LoadingPriority].fetch_add(1) == 0)
    {
        switch (m_LoadingPriority)
        {
        case AssetLoadingPriority::Minimal:     ECSHandle.emplace<ACLoadFlag<AssetLoadingPriority::Minimal  >>(); break;
        case AssetLoadingPriority::VeryLow:     ECSHandle.emplace<ACLoadFlag<AssetLoadingPriority::VeryLow  >>(); break;
        case AssetLoadingPriority::Low:         ECSHandle.emplace<ACLoadFlag<AssetLoadingPriority::Low      >>(); break;
        case AssetLoadingPriority::Standard:    ECSHandle.emplace<ACLoadFlag<AssetLoadingPriority::Standard >>(); break;
        case AssetLoadingPriority::High:        ECSHandle.emplace<ACLoadFlag<AssetLoadingPriority::High     >>(); break;
        case AssetLoadingPriority::VeryHigh:    ECSHandle.emplace<ACLoadFlag<AssetLoadingPriority::VeryHigh >>(); break;
        case AssetLoadingPriority::Critical:    ECSHandle.emplace<ACLoadFlag<AssetLoadingPriority::Critical >>(); break;
        default:                                FE_CORE_ASSERT(false, "Unrecognized AssetLoadingPriority value");
        }
    }
}
```


Access
AssetInterface
Generic interface to ECS
TextureObserver
TextureUser
ShaderObserver
ShaderUser
MaterialObserver
MaterialUser
AudioObserver
AudioUser
Observer
Interface to a concrete Asset
Public only methods specific to its asset type
Can be conceptualized as an asset itself, but actually is just an ECS handle (Asset ID + Registry ptr)
Treats Asset as const
User
Extends Observer adding methods modifying an Asset
AssetHandle< tnAsset >
AssetID + AssetLoadingPriority
.Observe()
.Use()
AssetObserver< tnAsset >
Common (templated) derivation of all observers
Acting like a shared lock guard
AssetUser< tnAsset >
Common (templated) derivation of all users
Acting like a unique lock guard

Synchronization





Observer()	User()
1. Lock Mutex A 2. Observers Count ++ 3. If 2. was 0->1 • Lock Mutex B 4. Unlock Mutex A	1. Lock Mutex A 2. Lock Mutex B
~Observer()	**~User()**
1. Observers Count -- 2. If 1. was 1->0 • Unlock Mutex B	1. Unlock Mutex B 2. Unlock Mutex A

Synchronized Runtime Access
Examples

```
void Renderer::Draw(
    const Ref<VertexBuffer>& vertexBuffer,
    const AssetObserver<Material>& materialObserver,
    const glm::mat4& transform,
    const glm::mat4& VPMatrix)
{
    FE_PROFILER_FUNC();

    auto& material_core = materialObserver.GetCoreComponent();
    auto sm_observer = AssetObserver<ShadingModel>(material_core.ShadingModelID);
    auto& sm_core = sm_observer.GetCoreComponent();
    auto shaderUser = AssetUser<Shader>(sm_core.ShaderID);

    shaderUser.Bind(s_ActiveGDI);

    shaderUser.UploadUniform(
        s_ActiveGDI,
        Uniform("u_ViewProjection", ShaderData::Type::Mat4),
        (void*)glm::value_ptr(VPMatrix)
    );
    shaderUser.UploadUniform(
        s_ActiveGDI,
        Uniform("u_Transform", ShaderData::Type::Mat4),
        (void*)glm::value_ptr(transform)
    );

    for (const auto& uniform : sm_core.Uniforms)
    {
        auto dataPointer = materialObserver.GetUniformValuePtr(material_core, uniform);
        shaderUser.UploadUniform(s_ActiveGDI, uniform, dataPointer);
    }
```

```
auto render_mesh_observer = render_mesh_component.RenderMesh.Observe();

auto& render_mesh_core = render_mesh_observer.GetCoreComponent();
auto material_observer = AssetObserver<Material>(render_mesh_core.MaterialID);
auto shaderID = AssetObserver<ShadingModel>(material_observer.GetCoreComponent().ShadingModelID).GetCoreComponent().ShaderID;
{
    auto shader_observer = AssetObserver<Shader>(shaderID);

    shader_observer.Bind(GDI);
    shader_observer.UploadUniform(GDI, Uniform("u_ViewProjection", ShaderData::Type::Mat4), VPmatrixPtr);
    shader_observer.UploadUniform(GDI, Uniform("u_ModelTransform", ShaderData::Type::Mat4), modelTransformPtr);
    shader_observer.UploadUniform(GDI, Uniform("u_EntityID", ShaderData::Type::UInt), &ID);
}

AssetObserver<Mesh>(render_mesh_core.MeshID).Draw(material_observer);
```